

CSE36301 Machine Learning Assignment 4

Deadline: Sunday, November 30, 2025, 11:59 PM

Introduction

In this assignment, you will explore four fundamental topics in deep learning and machine learning:

1. **Convolutional Neural Network (CNN):** Implement convolution operations from scratch using NumPy, experiment with PyTorch-based training on Tiny-ImageNet, and visualize feature maps and learned filters.
2. **Recurrent Neural Networks (RNNs):** Build Elman and Jordan RNN architectures, train them on time-series data (e.g., Sunspots dataset), and compare their predictive behavior with LSTM models.
3. **K-means Clustering:** Implement the complete K-means algorithm (including centroid updates and hard assignments) and evaluate clustering performance on synthetic 2D datasets.
4. **Gaussian Mixture Model (GMM):** Implement the Expectation–Maximization algorithm, perform soft assignments, and analyze the differences between K-means and GMM clustering.

The goal of this assignment is to understand how convolutional and recurrent architectures learn representations from data, and how unsupervised learning algorithms identify structure through clustering and probabilistic modeling. This assignment emphasizes both theoretical understanding and hands-on implementation. Students will implement core components of CNN, RNNs, K-means, and GMM *from scratch*, analyze their behavior through experiments.

Each section contains `TODO` comments that guide you step by step through the implementation process. You are encouraged to carefully read the comments, fill in the missing parts, and verify your results through visualization and performance comparison.

1 CNN

Dataset: Tiny-ImageNet ($64 \times 64 \times 3$)

Models: CNN (manual implementation by Pytorch)

Goal: To visually examine what kinds of features a CNN extracts by observing how its learned kernels (filters) respond to input images, either during training or after the model has been fully trained.

- T1.** Implement a 2D convolution operation using NumPy, and apply various predefined kernels (e.g., blur, sharpen, Sobel, Laplacian).
 - T2.** In the evaluation function, add functionality to store intermediate feature maps (history) for later visualization.
 - T3.** Analyze the feature maps (visualization) stored per epoch and evaluate how the learned representations change throughout training, focusing on the differences between the first layer and the last layer.
- Q1. (Extra credit)** If you change the hyperparameters, how do the feature maps change?

2 RNNs

Dataset: Sunspots dataset (309×1)

Goal: To explore how RNNs learn temporal dependencies in sequential data by implementing and comparing Elman RNNs, Jordan RNNs, and LSTMs.

- T1.** Implement the Elman RNN cycle, including the recurrent hidden-state update and output computation, for the given time-series dataset.
- T2.** Implement the Jordan RNN cycle by feeding the previous output back into the hidden state.
- T3.** Try adjusting the epochs and learning rate, and analyze which model shows the best performance.
- Q1.** Which model generally performs better, and why?

3 K-Means Clustering and GMM

Dataset: Gaussian mixture dataset

Goal: To understand the difference between hard assignment (K-means) and soft assignment (GMM) by implementing both algorithms from scratch, and to analyze which model provides better clustering performance.

- T1.** Implement the K-Means algorithm by initializing cluster centers randomly and completing both the assignment and update steps.
- T2.** Implement the GMM algorithm by randomly initializing its parameters and performing the Expectation–Maximization steps to estimate the mixture components.
 - (a) **Expectation step (E-step):** For each data point x_n and component k , compute the responsibility $r_{n,k}$:

$$r_{n,k} = \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k),$$

and normalize across all components:

$$r_{n,k} = \frac{r_{n,k}}{\sum_{j=1}^K r_{n,j}}.$$

where n is the data index and π_k is the mixture weight of component k .

- (b) **Maximization step (M-step):** Update the mixture parameters using the responsibilities from the E-step:

$$N_k = \sum_{n=1}^N r_{n,k}$$

$$\pi_k = \frac{N_k}{N}$$

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^N r_{n,k} x_n$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N r_{n,k} (x_n - \mu_k)(x_n - \mu_k)^\top$$

where N is the number of samples.

- T3.** Compare the clustering results of K-means and GMM, and analyze which model performs better. Discuss the reasons behind the performance differences (Try changing the random state).
- T4. (Extra credit)** If possible, extract features from a CNN, apply PCA for dimensionality reduction, and replace the classifier with a clustering model. Compare the performance of K-means and GMM on the reduced feature space and interpret the findings (v.s. CNN Classifier).

Submission Guidelines

- Your submission must be a `.zip` file containing the following files with the exact naming convention: `Assignment4_[YOUR_STUDENT_ID]_[COMPONENT].[EXTENSION]`
- Submit a Jupyter Notebook (`.ipynb`) including all code, results.
- Submit a written report including your analyses and all relevant plots (accuracy comparison, PCA visualization, learning curves, etc.).
- Code must be well-commented and reproducible.

Evaluation Criteria

- Correctness and completeness of implementation (50%)
- Depth of analysis and reasoning (25%)
- Clarity of visualizations and explanations (25%)

Contact

If you have any questions regarding this assignment, please email the TAs:

- Jinsik Bang: `bang@unist.ac.kr`
- Seokhoon Jeong: `shjd0246@unist.ac.kr`